

Riadenie a umelá inteligencia

Obsah

1	Zadanie 1	2
1.1	Znenie zadania 1	2
1.2	Vystup	2
2	Zadanie 2	6
2.1	Znenie zadania 2	6
2.2	Vystup	6
3	Zadanie 3	9
3.1	Znenie zadania 3	9
3.2	Outputs	14
4	Zadanie 4	16
4.1	Znenie zadania 4	16
4.2	Vystupy	16
4.3	Simulácia Van der Polovho oscilátora – <code>main.m</code>	16

1 Zadanie 1

1.1 Znenie zadania 1

- 1a) Je daný LDS reprezentovaný LDR:

$$y^{(3)}(t) + 6y^{(2)}(t) + 11y^{(1)}(t) + 6y(t) = u(t)$$

- 1b) Prepíšte LDR do Laplaceovej transformácie a získajte prenos:

$$F(s) = \frac{Y(s)}{U(s)}, \quad \text{pri PP} = 0$$

- 1c) Prepíšte LDR do substitučného kanonického tvaru, pričom $y(t) = x_1(t)$.
- 1d) Vyriešte LDR s využitím funkcie `ode45` v MATLABe a získajte numerické riešenie $y_N(t)$.
- 1e) Získajte analytické riešenie $y_A(t)$ pre LDR s využitím Symbolic Toolboxu v MATLABe.

1.2 Vystup

Matlab

```
1 %% LDR
2
3 clear;
4 clc;
5
6 %% 1a)
7 syms s
8 F_s = 1 / (s^3 + 6*s^2 + 11*s + 6);
9 disp('Prenosova_funkcia_F(s)=Y(s)/U(s):');
10 pretty(F_s)
11
12 %% 1b)
13 % x1 = y
14 % x2 = y'
15 % x3 = y''
16 % x1' = x2
17 % x2' = x3
18 % x3' = -6*x3 -11*x2 -6*x1 + u(t)
19
20 %% 1c)
21 u = @(t) ones(size(t))
22 syst = @(t, x) [x(2); x(3); -6*x(3) - 11*x(2) - 6*x(1) + u(t)];
23
24 x0 = [0; 0; 0];
25 tspan = [0 10];
```

```

26 [t, x] = ode45(syst, tspan, x0);
27 y_num = x(:,1);
28
29 %% 1d)
30 syms y(tsym) tsym
31 Dy = diff(y, tsym);
32 D2y = diff(y, tsym, 2);
33
34 LDR = diff(y, tsym, 3) + 6*diff(y, tsym, 2) + 11*diff(y, tsym, 1)
      + 6*y == 1;
35 conds = [y(0)==0, Dy(0)==0, D2y(0)==0];
36 y_analyt = dsolve(LDR, conds);
37 y_analyt = simplify(y_analyt);
38
39 disp('Analyticke riesenie y_A(t):');
40 pretty(y_analyt)
41
42
43
44 % Konverzia
45 y_fun = matlabFunction(y_analyt, 'Vars', tsym);
46 y_ana = y_fun(t);
47
48 %% 1e) Simulink simulacia LDR
49 a0 = 6;
50 a1 = 11;
51 a2 = 6;
52
53 out = sim('simulink_LDR.slx', ...
54         'SimulationMode', 'normal', ...
55         'StartTime', '0', ...
56         'StopTime', '10', ...
57         'SrcWorkspace', 'current');
58
59 tS = out.simout.Time;
60 yS = out.simout.Data(:, end);
61
62 %% 1f) Porovnanie
63 figure;
64
65 subplot(3,1,1);
66 plot(t, y_num, 'b', 'LineWidth', 2); hold on;
67 plot(t, y_ana, 'r--', 'LineWidth', 2);
68 title('Porovnanie y_N(t) a y_A(t)');
69 legend('Numericke (ode45)', 'Analyticke (dsolve)');
70 xlabel('Cas [s]');
71 ylabel('y(t)');
72 grid on;
73
74 subplot(3,1,2);
75 plot(t, y_num, 'b', 'LineWidth', 2); hold on;

```

```

76 plot(tS, yS, 'r--', 'LineWidth', 2);
77 title('Porovnanie y_N(t) a y_S(t)');
78 legend('Numericke (ode45)', 'Simulink');
79 xlabel('Cas [s]');
80 ylabel('y(t)');
81 grid on;
82
83 subplot(3,1,3);
84 plot(tS, yS, 'b', 'LineWidth', 2); hold on;
85 plot(t, y_ana, 'r--', 'LineWidth', 2);
86 title('Porovnanie y_S(t) a y_A(t)');
87 legend('Simulink', 'Analyticke (dsolve)');
88 xlabel('Cas [s]');
89 ylabel('y(t)');
90 grid on;

```

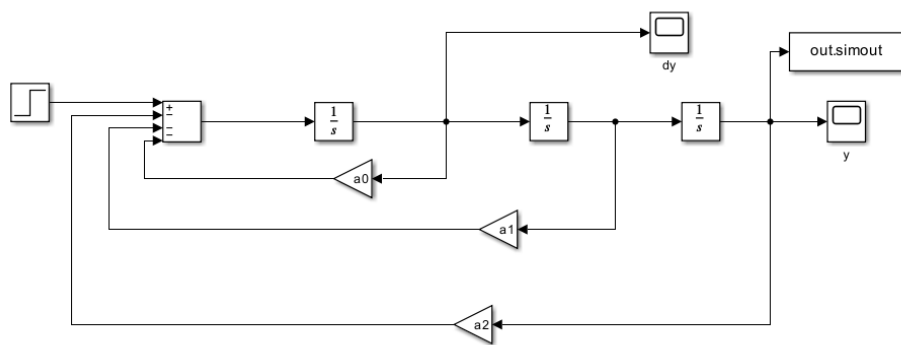
Prenosová funkcia $F(s) = Y(s) / U(s)$:

$$\frac{1}{s^3 + 6s^2 + 11s + 6}$$

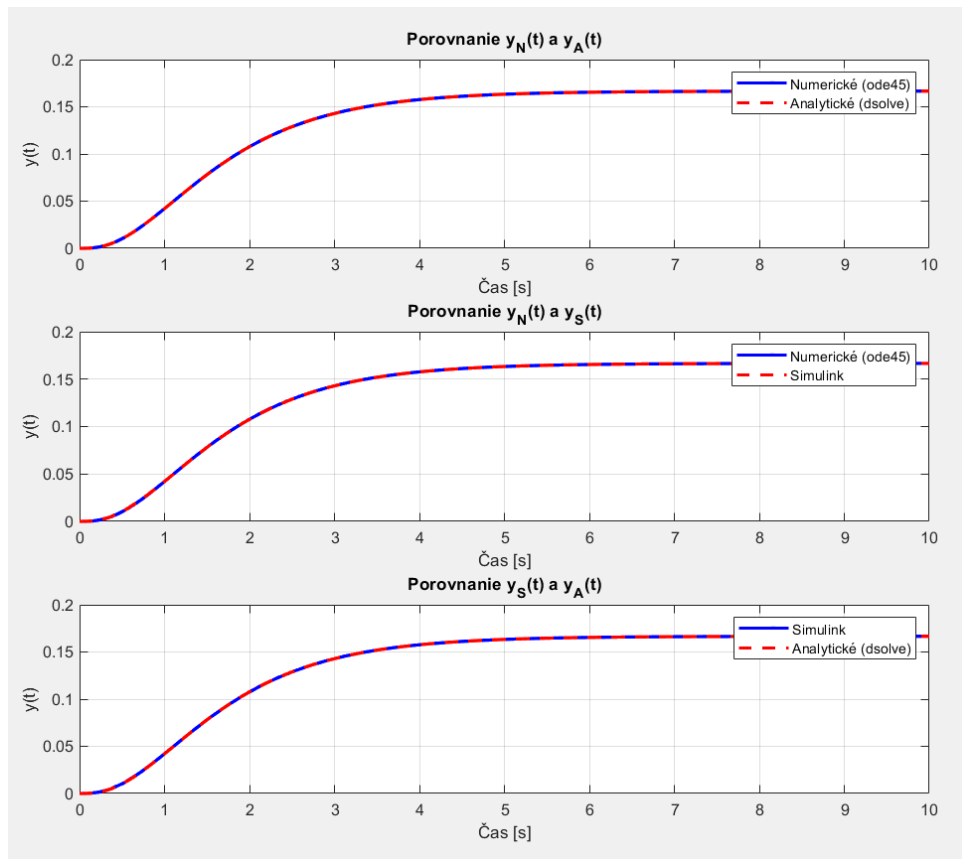
Analytické riešenie $y_A(t)$:

$$\frac{\exp(-3 \text{ tsym}) (\exp(\text{tsym}) - 1)}{6}$$

Obr. 1: Matlab vystup



Obr. 2: simulinkLDR.slx



Obr. 3: Porovnanie

2 Zadanie 2

2.1 Znenie zadania 2

Je daný lineárny dynamický systém (LDS) reprezentovaný prenosom $F_p(s)$ z PR1 (napr. RLC obvod).

2a) Pre prenos $F_p(s)$ navrhnite PID riadenie $\{K, T_i, T_d\}$ a overte ho v riadiacej štruktúre v Simulinku.

Využite metódu MOM (metóda otvorenej slučky) a ZN (Ziegler–Nichols).

V prípade potreby možno využiť aj metódu Naslina.

2b) Implementujte funkcie pre návrh PID regulátora pomocou metód MOM, ZN (a Naslina) ako samostatné funkcie.

Výsledky simulácií uložte a znázornite graficky:

- Výstupy systému: $y_{\text{MOM}}(t)$ a $y_{\text{ZN}}(t)$
- Regulačné zásahy: $u_{\text{MOM}}(t)$ a $u_{\text{ZN}}(t)$
- Vstupné signály: $w(t) \neq 0, z = 0$ a $w(t) = 0, z \neq U$

2c) Vypočítajte koeficienty PSD regulátora pre rýchlostný algoritmus:

$$\{q_0, q_1, q_2\}$$

pre vhodne zvolenú vzorkovaciu periódu T_{vz} .

2d) Overte rýchlostný PSD algoritmus pre diskretný prenos $F_p(z)$ v riadiacej štruktúre v Simulinku.

2.2 Vystup

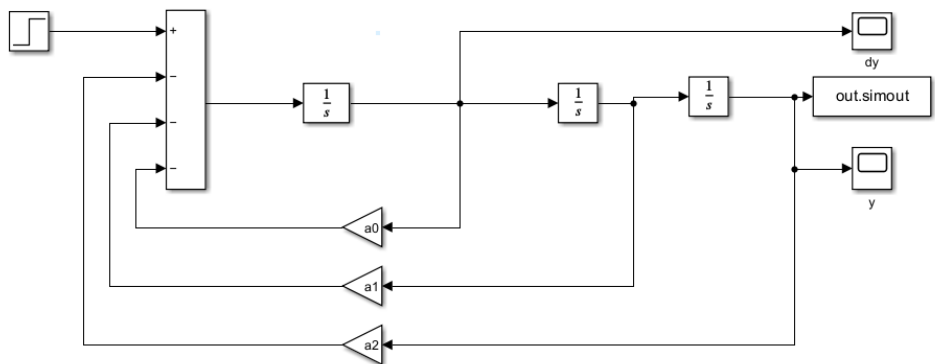
Matlab

```
1 clear; clc;
2
3 Fp = tf(1, [1 6 11 6]);
4
5 K = 1;
6 Tu = 0.5;
7 Tr = 4.0;
8
9 Kp_MOM = 1.2 * Tr / (K * Tu);
10 Ti_MOM = 2 * Tu;
11 Td_MOM = 0.5 * Tu;
12
13 Kcrit = 3.5;
14 Tcrit = 1.8;
15
16 Kp_ZN = 0.6 * Kcrit;
17 Ti_ZN = Tcrit / 2;
18 Td_ZN = Tcrit / 8;
```

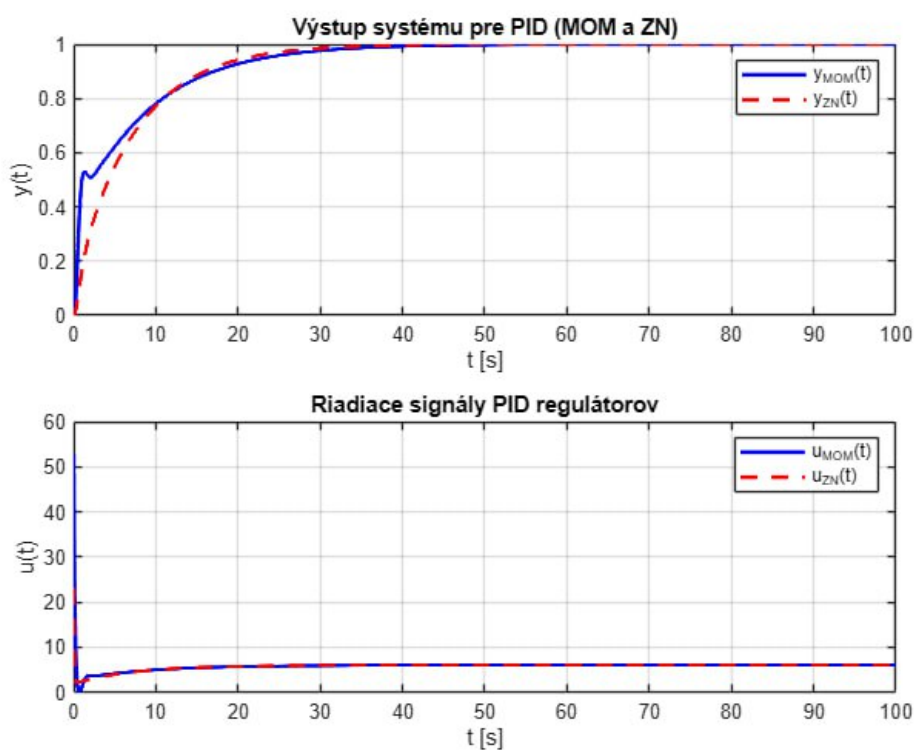
```

19
20 N = 1;
21
22 PID_MOM = pidstd(Kp_MOM, Kp_MOM / Ti_MOM, Kp_MOM * Td_MOM, N);
23 PID_ZN = pidstd(Kp_ZN, Kp_ZN / Ti_ZN, Kp_ZN * Td_ZN, N);
24
25 T = 0:0.01:100;
26 w = ones(length(T), 1);
27
28 Sys_MOM = feedback(PID_MOM * Fp, 1);
29 Sys_ZN = feedback(PID_ZN * Fp, 1);
30 yMOM = lsim(Sys_MOM, w, T);
31 yZN = lsim(Sys_ZN, w, T);
32 eMOM = w - yMOM;
33 eZN = w - yZN;
34
35 assert(size(eMOM,1) == length(T), 'Dimension_error_eMOM');
36 assert(size(eZN,1) == length(T), 'Dimension_error_eZN');
37
38 uMOM = lsim(PID_MOM, eMOM, T);
39 uZN = lsim(PID_ZN, eZN, T);
40
41 figure;
42
43 subplot(2,1,1);
44 plot(T, yMOM, 'b', T, yZN, 'r--', 'LineWidth', 1.8);
45 legend('y_{MOM}(t)', 'y_{ZN}(t)');
46 xlabel('t[s]'); ylabel('y(t)');
47 title('System_output_for_PID(MOM_and_ZN)');
48 grid on;
49
50 subplot(2,1,2);
51 plot(T, uMOM, 'b', T, uZN, 'r--', 'LineWidth', 1.8);
52 legend('u_{MOM}(t)', 'u_{ZN}(t)');
53 xlabel('t[s]'); ylabel('u(t)');
54 title('Control_signals_for_PIDcontrollers');
55 grid on;
56
57 Ts = 0.1;
58 K = Kp_ZN;
59 Ti = Ti_ZN;
60 Td = Td_ZN;
61
62 q0 = K * (1 + Ts/Ti + Td/Ts);
63 q1 = K * (-1 - 2*Td/Ts);
64 q2 = K * (Td/Ts);
65
66 fprintf('\nPSD_coefficients_for_velocity_algorithm(Tvz=%.2f_s)\n', Ts);
67 fprintf('q0=%.4f\nq1=%.4f\nq2=%.4f\n', q0, q1, q2);

```

Obr. 4: simulink2.slx



Obr. 5: out

3 Zadanie 3

3.1 Znenie zadania 3

Je daný model pre servomotor:

$$J \frac{d^2 \psi(t)}{dt^2} + B \frac{d\psi(t)}{dt} = K u(t) \quad (1)$$

Prenosová funkcia:

$$F(s) = \frac{\Psi(s)}{U(s)} = \frac{K}{Js^2 + Bs} \quad (2)$$

Po úprave:

$$F(s) = \frac{K}{s(Ts + 1)} \quad (3)$$

kde $K_v = \frac{K}{B}$, $T = \frac{J}{B}$

Popisy:

- K_v – výstupná konštanta
- T – časová konštanta
- $\psi(t)$ – poloha hriadeľa
- $u(t)$ – napätie / akčný zásah
- J – moment zotrvačnosti
- B – viskózne trenie

Zadanie:

1. Pre servo s prenosom $F(s) = \frac{K_v}{s(Ts+1)}$ navrhните riadenie priamou a nepriamou metódou syntézy.
2. **Nepriama metóda:** PD algoritmus s využitím Graham-Lathrop (G-L). Určíte $\psi(t)$ (výstup), $u(t)$ (akčný zásah) pre riadenie na požadovanú polohu (Simulink/RT).
3. **Priama metóda:** Priama syntéza, použite polohový predpisovač (PP) alebo riadiacu slučku a otestujte v Simulinku.
4. Ak nebude fungovať žiaden algoritmus priamej/nepriamej syntézy, skúste nastaviť $D = B a_{reg}$.

Matlab

```
1 clear all; close all; clc;
2
3 Kj = 5;
4 T = 0.15;
5 num = Kj;
6 den = [T, 1, 0];
7 Gp = tf(num, den);
8 [Kp, Kd] = GL(Gp);
9
10 assignin('base','Kp', Kp);
11 assignin('base','Kd', Kd);
12
13 simOut = sim('Motor.slx');
14 gl_t = simOut.simout.t_.Data;
15 gl_y = simOut.simout.y_t_.Data;
16 gl_w = simOut.simout.w_t_.Data;
17 gl_u = simOut.simout.u_z_t_.Data;
18 gl_e = simOut.simout.e_t_.Data;
19
20 figure;
21 subplot(3,1,1);
22 plot(gl_t, gl_y, 'k','LineWidth',1.5); hold on
23 stairs(gl_t, gl_w, 'r','LineWidth',1.2); hold off
24 grid on
25 title('System output y(t)'), ylabel('y(t)')
26 legend({'y(PD)', 'w'}, 'Location','best')
27
28 subplot(3,1,2);
29 plot(gl_t, gl_u, 'r','LineWidth',1.5);
30 grid on
31 title('Control input u(t)'), ylabel('u(t)')
```

```
1 function [Kp, Kd] = GL(Gp)
2     [num, den] = tfdata(Gp, 'v');
3
4     b0 = num(end);
5
6     a2 = den(1);
7     a1 = den(2);
8     a0 = den(3);
9
10    omega0 = 4;
11
12    Kp = (omega0^2 * a2 - a0) / b0;
13    Kd = (2 * omega0 * a2 - a1) / b0;
14 end
```

```
1 Ku = 5;
2 T = 0.15;
3 sim_stop_time = 10;
```

```

4
5 disp('Start_PID_simulation');
6 pid_model_name = 'servo';
7 open_system(pid_model_name);
8
9 Kp_pid = 2.0;
10 Ki_pid = 0.5;
11 Kd_pid = 0.1;
12
13 set_param([pid_model_name '/PID_Controller'], 'P', num2str(Kp_pid
14 ));
15 set_param([pid_model_name '/PID_Controller'], 'I', num2str(Ki_pid
16 ));
17 set_param([pid_model_name '/PID_Controller'], 'D', num2str(Kd_pid
18 ));
19 set_param([pid_model_name '/Servomechanism'], 'Numerator',
20 num2str(Ku));
21 set_param([pid_model_name '/Servomechanism'], 'Denominator', ['['
22 num2str(T) '1 0']]);
23 set_param(pid_model_name, 'StopTime', num2str(sim_stop_time));
24
25 sim(pid_model_name);
26
27 figure('Name', 'PID_Regulator_Response');
28 subplot(2,1,1);
29 plot(phi_output_pid.time, phi_output_pid.signals.values, 'b', '
30 DisplayName', 'Position_phi');
31 hold on;
32 if exist('w_t', 'var') && isstruct(w_t) && isfield(w_t, 'time')
33 && isfield(w_t, 'signals')
34     plot(w_t.time, w_t.signals.values, 'r--', 'DisplayName', '
35 Reference_w');
36 end
37 hold off;
38 xlabel('Time(s)');
39 ylabel('Amplitude');
40 legend('show');
41 grid on;
42
43 subplot(2,1,2);
44 plot(u_control_pid.time, u_control_pid.signals.values);
45 title('Control_signal(u) for PID');
46 xlabel('Time(s)');
47 ylabel('Amplitude');
48 grid on;

```

```

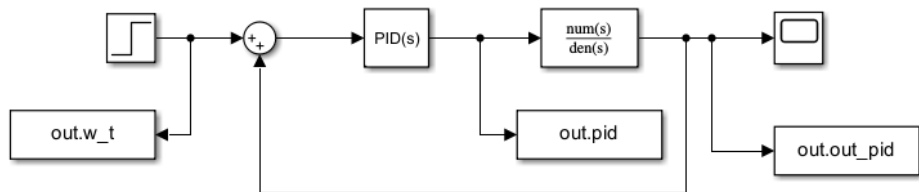
1 Ku = 5;
2 T = 0.15;
3 sim_stop_time = 10;
4
5 disp('Start_PID_simulation');
6 pid_model_name = 'servo';
7 open_system(pid_model_name);
8
9 Kp_pid = 2.0;
10 Ki_pid = 0.5;
11 Kd_pid = 0.1;
12
13 set_param([pid_model_name '/PID_Controller'], 'P', num2str(Kp_pid
    ));
14 set_param([pid_model_name '/PID_Controller'], 'I', num2str(Ki_pid
    ));
15 set_param([pid_model_name '/PID_Controller'], 'D', num2str(Kd_pid
    ));
16 set_param([pid_model_name '/Servomechanism'], 'Numerator',
    num2str(Ku));
17 set_param([pid_model_name '/Servomechanism'], 'Denominator', ['[ '
    num2str(T) ' 1 0']]);
18 set_param(pid_model_name, 'StopTime', num2str(sim_stop_time));
19
20 sim(pid_model_name);
21
22 figure('Name', 'PID_Response');
23 subplot(2,1,1);
24 plot(phi_output_pid.time, phi_output_pid.signals.values, 'b', '
    DisplayName', 'Position_phi');
25 hold on;
26 if exist('w_t', 'var') && isstruct(w_t) && isfield(w_t, 'time')
    && isfield(w_t, 'signals')
27     plot(w_t.time, w_t.signals.values, 'r--', 'DisplayName', '
        Reference_w');
28 end
29 hold off;
30 title('Servo_position_response_with_PID');
31 xlabel('Time(s)');
32 ylabel('Amplitude');
33 legend('show');
34 grid on;
35
36 subplot(2,1,2);
37 plot(u_control_pid.time, u_control_pid.signals.values);
38 title('Control_signal(u)_for_PID');
39 xlabel('Time(s)');
40 ylabel('Amplitude');
41 grid on;
42

```

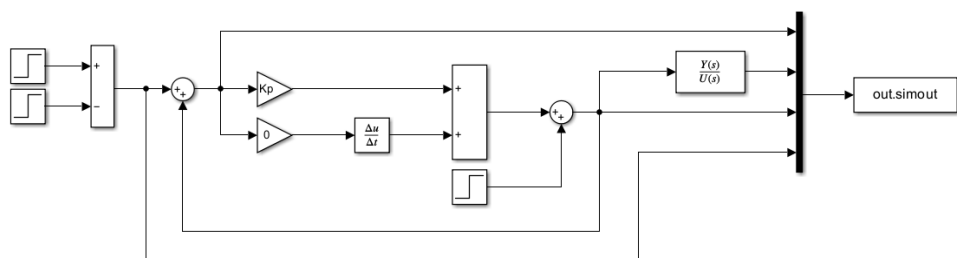
```

43 fprintf('\n');
44
45 pp_model_name = 'servo_pole_placement';
46 open_system(pp_model_name);
47 k1_pp = 0.24;
48 k2_pp = -0.08;
49 gain_x2_term_val = -20/3;
50 gain_u_term_val = 100/3;
51
52 set_param([pp_model_name '/Gain_x2_term'], 'Gain', num2str(
    gain_x2_term_val));
53 set_param([pp_model_name '/Gain_u_term'], 'Gain', num2str(
    gain_u_term_val));
54 set_param([pp_model_name '/Gain_k1'], 'Gain', num2str(k1_pp));
55 set_param([pp_model_name '/Gain_k2'], 'Gain', num2str(k2_pp));
56
57 A = [0, 1; 0, -1/T];
58 B_plant = [0; Ku/T];
59 C = [1, 0];
60 D = 0;
61
62 K_current_simulink = [k1_pp, k2_pp];
63 A_cl = A - B_plant * K_current_simulink;
64 poles_cl = eig(A_cl);
65
66 figure('Name', 'Pole_Placement_Response');
67 subplot(2,1,1);
68 plot(phi_output_pp.time, phi_output_pp.signals.values, 'b', '
    DisplayName', 'Position_phi');
69 hold on;
70 if exist('w_t', 'var') && isstruct(w_t) && isfield(w_t, 'time')
    && isfield(w_t, 'signals')
71     plot(w_t.time, w_t.signals.values, 'r--', 'DisplayName', '
        Reference_w');
72 end
73 hold off;
74 title('Servo_position_response_with_Pole_Placement');
75 xlabel('Time(s)');
76 ylabel('Amplitude');
77 legend('show');
78 grid on;
79
80 subplot(2,1,2);
81 plot(u_control_pp.time, u_control_pp.signals.values);
82 title('Control_signal(u)for_Pole_Placement');
83 xlabel('Time(s)');
84 ylabel('Amplitude');
85 grid on;

```

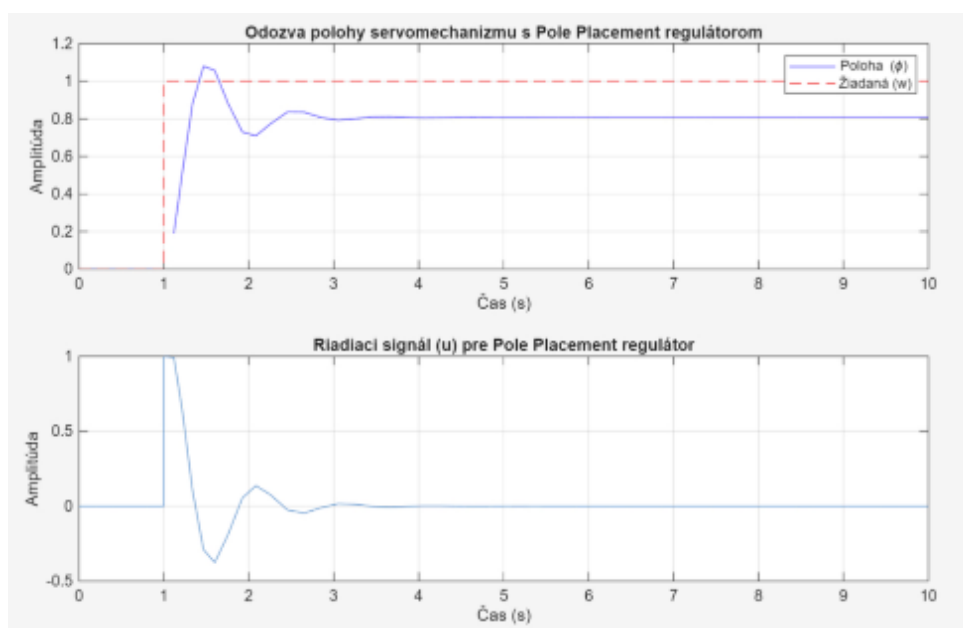


Obr. 6: simulink3.slx

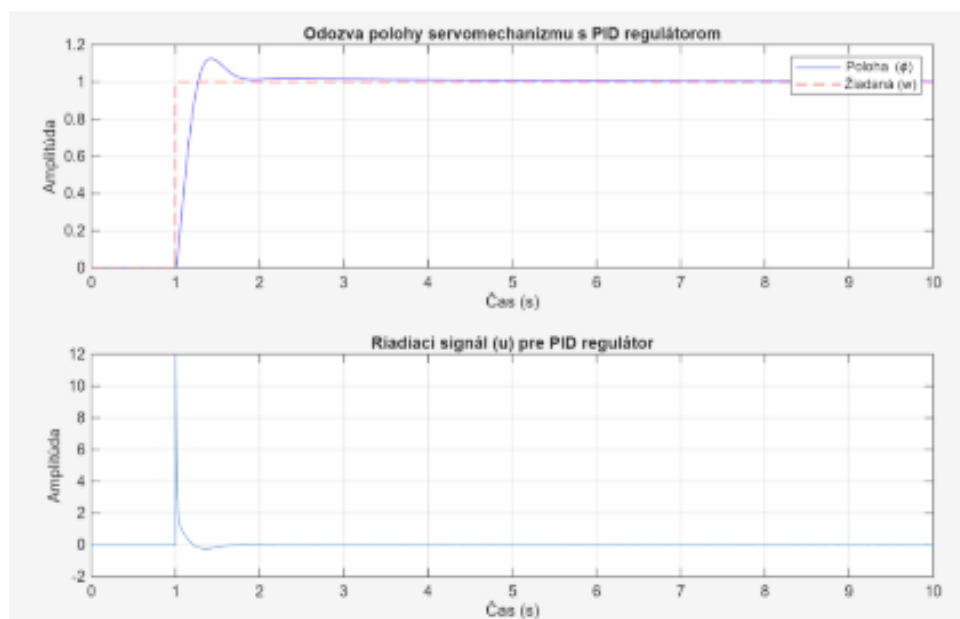


Obr. 7: simulink31.slx

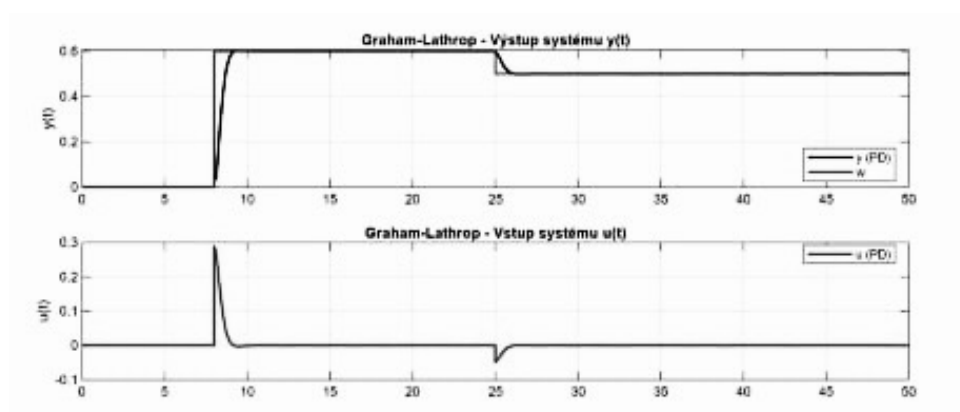
3.2 Outputs



Obr. 8: out



Obr. 9: out1



Obr. 10: out2

4 Zadanie 4

4.1 Znenie zadania 4

- 4a) Prepiste NDS do SKT
- 4b) F-cia pre numericke riesenie NDS
- 4c) modely NDS zapiste do substitucneho kanonickeho tvaru
- 4d) implementujte matematicky model NDS reprezentovany NDR do prostredia Simulink a vykonajte simulacie s vyuzitim zvoleneho solvera, kroku a casu simulacie

4.2 Vystupy

Matlab

4.3 Simulácia Van der Polovho oscilátora – main.m

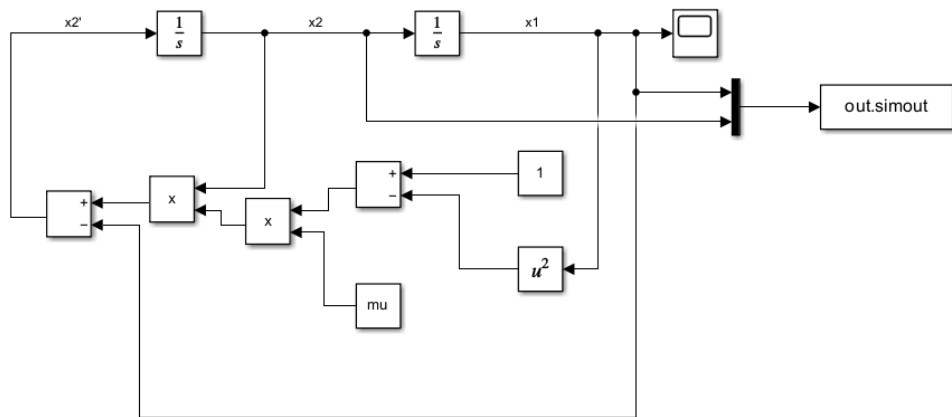
```
1 mu = 1;
2 tspan = [0 20];
3 x0 = [2; 0];
4 h = 0.1;
5
6 [t1, X1] = ode45(@(t, x) van_der_pol(t, x, mu), tspan, x0);
7
8 out = sim("model.slx");
9
10 hold on
11 plot(t1, X1(:,1), 'b', 'LineWidth', 1.5);
12 plot(out.simout.Time, out.simout.Data(:, 1), 'g*');
13 hold off
14
15 xlabel('Time');
16 ylabel('x_1');
17 title('Van der Pol Oscillator');
18 legend(["ode45" "Simulink"])
19 grid on;
```

Listing 1: Hlavný skript main.m

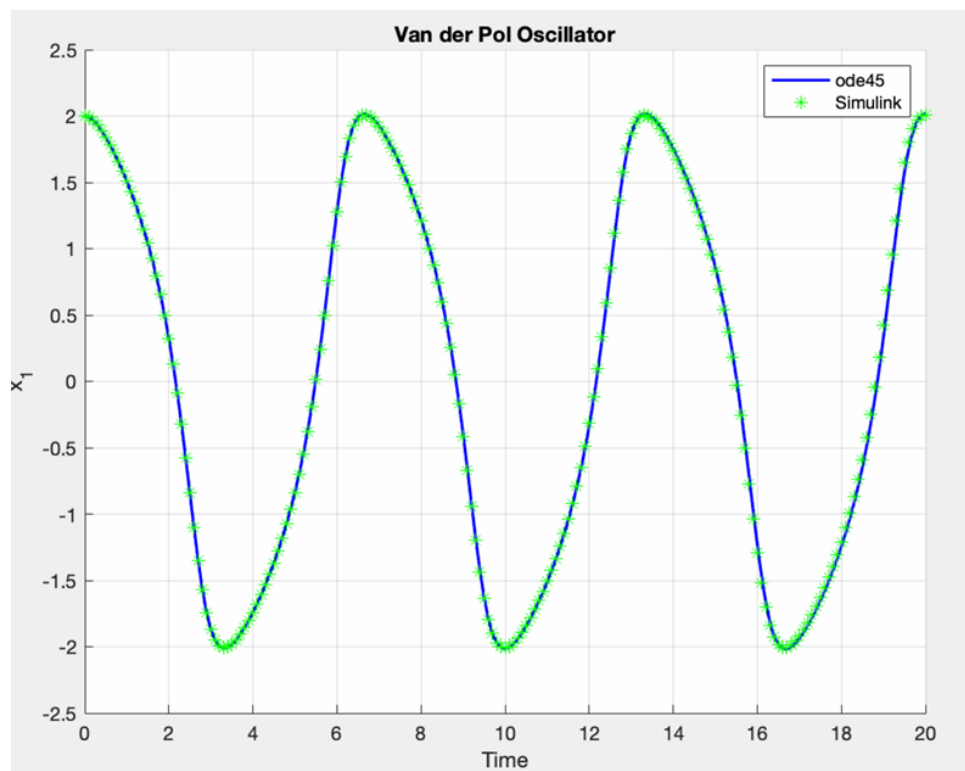
Dynamika systému – van_der_pol.m

```
1 function dxdt = van_der_pol(t, x, mu)
2     dxdt = zeros(2,1);
3     dxdt(1) = x(2);
4     dxdt(2) = mu * (1 - x(1)^2) * x(2) - x(1);
5 end
```

Listing 2: Funkcia van_der_pol.m



Obr. 11: simulink4.slx



Obr. 12: ode45 VS Van der Pol